

Project BRD: OmniRoute Smart Logistics Engine

1. Introduction

The **OmniRoute** system is an enterprise-grade Data Warehouse (DWH) solution designed to ingest, process, and analyze high-velocity telemetry data from a global fleet of delivery vehicles. The system integrates batch data from AWS S3 (vehicle registries, assignment logs, and maintenance schedules) with real-time IoT streams from Kafka (GPS and sensor data) to monitor driver safety and asset health.

2. Business Objectives

The system aims to solve the following logistics challenges:

- **Asset Lifecycle Management:** Maintain an accurate, historical record of vehicle assignments and driver transitions using SCD Type 2 logic.
- **Operational Efficiency:** Detect vehicles with abnormal fuel consumption patterns compared to fleet baselines.
- **Safety Compliance:** Real-time monitoring of driver behavior (speeding and geofencing) to mitigate risk.
- **Financial Accountability:** Automate driver rate deductions based on safety violations and manage monthly cooldown periods.

3. System Requirements

3.1 Data Sources & Frequency

Data Asset	Format	Frequency	Description
Vehicle Registry	CSV	Daily	Master list of all fleet vehicles including vin, model, and mfg_year.
Vehicle Assignment	CSV	Daily (Incremental)	History of which driver is assigned to which vehicle, including daily_rate.
Maintenance Logs	CSV	Yearly (Jan 1st)	Scheduled downtime and mandatory service dates for the fleet.

Fuel Transactions	CSV	Daily (05:00 UTC)	Logs of fuel quantity, cost, and odometer readings.
Telemetry Stream	JSON	Real-time (Kafka)	Live sensor data: vin, speed, rpm, and location_coords.
Restricted Zones	JSON	Static/Ad-hoc	A "marked" list of GPS coordinates defining no-go zones.

3.2 Advanced Data Processing Logic

- **Asset History (SCD Type 2):**
 - Create an incremental table tracking vin, start_date, end_date, driver_id, daily_rate, and status.
 - **Conflict Resolution:** If duplicate records exist for the same VIN and timeframe, retain only the record with the **highest daily rate**.
 - **Continuity:** When a new assignment record is ingested, the previous record for that VIN must be closed (end date = new start date) and marked as "ARCHIVED".
 - **Active Status:** Records without an end date are marked as "IN-TRANSIT".
- **Fuel Efficiency Auditing:**
 - Identify vehicles where "Fuel Consumed vs. Distance" exceeds the baseline by **12%** (excluding weekends and maintenance days).
 - Generate a daily report by 05:00 UTC showing the count of "IN-TRANSIT" vehicles categorized by model.

3.3 Safety & Streaming Requirements

- **Violation Detection:** A streaming pipeline must flag events where speed exceeds 110 km/h or coordinates intersect with restricted_zones.json.
- **Penalty System:** * Each flagged event counts as one "Safety Strike".
 - Deduct **5% from the Driver's Daily Rate** for each strike, represented as a separate calculated column in the resultant table.

- **Cooldown & Suspension:**
 - On the 1st of every month at 05:00 UTC, reset strikes and restore rates for eligible drivers.
 - Drivers who accumulate **10 strikes** are toggled to "SUSPENDED" status and are **excluded** from the monthly cooldown.

4. Technical Architecture

The solution must be built exclusively on **AWS** following DWH principles for fault tolerance and scalability:

- **Storage:** S3 (Data Lake) for raw, silver, and gold layers.
- **Compute:** Spark/Glue for batch ETL; Flink or Spark Streaming for Kafka telemetry.
- **Database:** If required, an RDBMS (like PostgreSQL) can be hosted on an **EC2 instance** for final reporting.
- **Orchestration:** Airflow to manage dependencies and daily scheduling at 05:00 UTC.

5. Success Criteria

- Successful ingestion and deduplication of vehicle assignment logs from S3.
- Automated generation of monthly rate-deduction reports in text format for fleet managers.
- Zero-duplicate processing in the event of job failure (Idempotency).
- Functional real-time system tracking active safety strikes across the fleet.

6. Reporting & Analytics Layer

This section extends the OmniRoute Smart Logistics Engine by defining the **reporting, analytics, and consumption layer**, ensuring that all processed data can be operationalized for business, compliance, and finance stakeholders.

6.1 Reporting Objectives

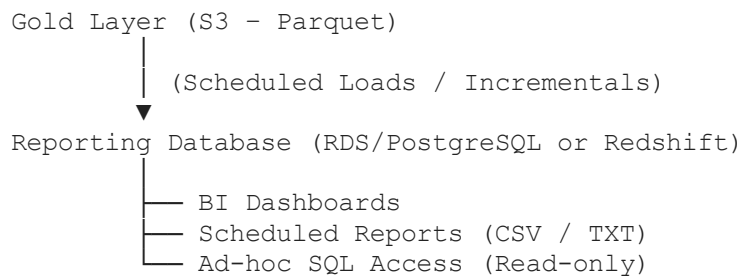
The Reporting Layer is designed to:

- Provide timely, consistent, and auditable views of fleet operations
- Enable daily and monthly management reporting without querying raw or streaming systems
- Serve as a single source of truth for finance, operations, and compliance teams
- Decouple analytical workloads from ETL and streaming pipelines

6.2 Reporting Architecture Overview

The Reporting Layer sits on top of the Gold Layer and exposes curated datasets for analytics tools and downstream consumers.

Architecture Flow



6.3 Reporting Storage & Technology

Component	Technology	Purpose
Reporting DB	PostgreSQL on EC2 / Amazon RDS (or Redshift if scale increases)	Final query-optimized reporting layer
Data Transport	Spark/Glue	Batch load from Gold S3 tables
Query Access	SQL	Standardized reporting access
BI Tools	QuickSight / Power BI / Tableau (optional)	Visualization & executive reporting

6.4 Reporting Data Models

The following **report-ready tables** are created in the Reporting Layer.

6.4.1 Fleet Assignment History Report

Source: gold.asset_history_scd2

Purpose: Historical traceability of drivers, vehicles, and rate changes.

Key Columns

- vin

- driver_id
- start_date
- end_date
- daily_rate
- status
- region

Usage

- Driver audits
- Legal & compliance investigations
- Historical payroll validation

6.4.2 Active Fleet Snapshot Report

Source: gold.active_fleet_snapshot

Refresh Frequency: Daily @ 05:00 UTC

Purpose: Executive-level fleet utilization monitoring.

Column	Description
model	Vehicle model
no_of_active_vehicles	Count of IN-TRANSIT vehicles
snapshot_time	Snapshot timestamp

6.4.3 Fuel Efficiency Audit Report

Source: gold.fuel_efficiency_audit

Purpose: Identify abnormal fuel consumption and potential misuse.

Key Fields

- vin
- model
- audit_date
- km_per_liter
- baseline_kmpl
- status (FLAGGED / OK)

Business Use

- Cost control
- Driver performance review
- Maintenance planning

6.4.4 Driver Safety & Penalty Report

Source: gold.driver_safety_status

Purpose: Near-real-time and monthly payroll impact analysis.

Field	Description
driver_id	Unique driver identifier
base_rate	Original daily rate

Field	Description
strike_count	Active month strike count
current_adjusted_rate	Penalized daily rate
status	ACTIVE / SUSPENDED
month	Reporting month

6.5 Scheduled & Regulatory Reports

6.5.1 Monthly Driver Rate Deduction Report

Report Includes

- Driver ID
- Total strikes for the month
- Total rate deductions
- Final payable daily rate
- Suspension status (if applicable)

6.5.2 Safety Compliance Summary

Purpose: Monitor safety trends and high-risk behavior.

Metrics

- Total violations by day
- Top 10 drivers by strike count
- Restricted zone breaches
- Speed violation counts

To provide a clear baseline, here is the detailed breakdown of each data asset for the **OmniRoute** system, including expected schemas and sample records.

3.1 Data Assets: Detailed Schema & Samples *(this is detailed explanation for point no 3 mentioned above)*

1. Vehicle Registry (vehicle_registry.csv)

This serves as the master dimension table. It is delivered as a full snapshot once daily.

- **Schema:** vin (String), model (String), mfg_year (Integer), fuel_type (String).
- **Frequency:** Daily at 00:00 UTC.
- **Sample:**

Vin	Model	mfg_year	fuel_type
1HGBH225	Freightliner M2	2022	Diesel

3FA6P0HD	Volvo VNL	2023	LNG
----------	-----------	------	-----

2. Vehicle Assignment (vehicle_assignment.csv)

This file tracks driver transitions and pay rates. It includes daily incremental updates.

- **Schema:** vin (String), driver_id (String), start_timestamp (Unix Timestamp), end_timestamp (Unix Timestamp/Null), daily_rate (Float), region (String).
- **Frequency:** Daily at 00:00 UTC.
- **Data Handling:** Convert Unix timestamps to date format and ensure record continuity.
- **Sample:**

Vin	driver_id	start_timestamp	end_timestamp	daily_rate
1HGBH225	DRV_902	1712818800	1713078000	450.00
1HGBH225	DRV_902	1713078000		485.00

3. Maintenance Logs (maintenance_schedules.csv)

Represents the mandatory downtime for vehicles. These are "over and above" standard working days.

- **Schema:** vin (String), service_date (Date), service_type (String).
- **Frequency:** Yearly on Jan 1st.
- **Sample:**

Vin	service_date	service_type
1HGBH225	2026-06-15	Engine Overhaul
3FA6P0HD	2026-08-20	Tire Rotation

4. Fuel Transactions (fuel_transactions.csv)

Logs the actual fuel usage used to calculate the 12% efficiency threshold.

- **Schema:** transaction_id (String), vin (String), fuel_liters (Float), odometer_reading (Float), timestamp (UTC).
- **Frequency:** Daily at 07:00 UTC.
- **Sample:**

transaction_id	vin	fuel_liters	odometer_reading	timestamp
TXN_550	1HGBH225	120.5	45200.0	2026-04-14 06:45:00

5. Telemetry Stream (Kafka Stream)

Real-time JSON objects used for safety flagging.

- **Schema:** vin (String), driver_id (String), speed (Integer), lat (Float), long (Float), event_timestamp (Kafka Timestamp).

- **Frequency:** Real-time.

- **Sample:**

```
JSON
{
  "vin": "1HGBH225",
  "driver_id": "DRV_902",
  "speed": 115,
  "lat": 28.6139,
  "long": 77.2090
}
```

6. Restricted Zones (restricted_zones.json)

A reference file containing coordinates that trigger a "Safety Strike" if breached.

- **Schema:** zone_name (String), min_lat (Float), max_lat (Float), min_long (Float), max_long (Float).

- **Frequency:** Static/Updated ad-hoc.

- **Sample:**

```
JSON
[
  {
    "zone_name": "High_Risk_Pass_A",
    "min_lat": 34.05,
    "max_lat": 34.10,
    "min_long": -118.25,
    "max_long": -118.20
  }
]
```

To help you implement the **OmniRoute** logic, here is a breakdown of the processing scenarios with examples.

1. Asset History: SCD Type 2 & Continuity

This logic ensures we track every driver who has ever touched a vehicle without losing historical data.

Scenario: The Driver Swap

Suppose **Vehicle-A** is currently being driven by **Driver-1**. On April 15th, **Driver-2** takes over.

Current State (Table):

Vin	driver_id	start_date	end_date	daily_rate	status
VIN-100	DRV-001	2026-04-01	NULL	500	IN-TRANSIT

New Incremental File Received:

- vin: VIN-100, driver_id: DRV-002, start_date: 2026-04-15, daily_rate: 550

Final State after Processing: The system must "close" the old record and "open" the new one.

- **Update Row 1:** Set end_date to 2026-04-15 and status to **ARCHIVED**.
- **Insert Row 2:** New record with start_date 2026-04-15, end_date as NULL, and status as **IN-TRANSIT**.

2. Conflict Resolution: The "Highest Rate" Rule

In data engineering, upstream systems often send duplicate or corrected records in the same batch.

Scenario: Overlapping Records

Your daily S3 folder receives two records for the same vehicle for the same day.

- **Record A:** VIN-200, Driver-X, Rate: **400**
- **Record B:** VIN-200, Driver-Y, Rate: **600**

Processing Logic: The developer must use a window function (like ROW_NUMBER()) partitioned by vin and start_date ordered by daily_rate DESC).

- **Result:** Record A is discarded. Record B is ingested into the DWH.

3. Efficiency Auditing: Fuel vs. Distance

This identifies "fuel abuse" by comparing actual consumption against a fleet baseline.

Scenario: Identifying an Outlier

- **The Baseline:** The average fuel consumption for "Freightliner M2" models is **5.0 km/Liter**.
- **The Threshold:** 12% over baseline means any vehicle doing worse than **4.4 km/Liter** (5.0 - 12%) is flagged.

Calculation Data Points:

- **Fuel Transactions:** Vehicle VIN-300 consumed **100 Liters**.
- **Distance:** Odometer went from 10,000 to 10,400 (**400 km traveled**).
- **Result:** 400km / 100L = **4.0 km/Liter**.
- **Audit Result:** Since 4.0 is less than the 4.4 threshold, this vehicle is flagged in the 05:00 UTC report.

Critical Exclusion: If April 12th was a Sunday (weekend) or the vehicle was in the maintenance_schedules.csv for that day, that day's fuel data is **ignored** to ensure the driver isn't penalized for idling at a workshop.

4. Reporting: Active Fleet Count

This is a simple aggregation used by management to see fleet distribution.

Scenario: Morning Snapshot

At 05:00 UTC, the system scans the Asset History table for all records where status = 'IN-TRANSIT'.

Example Output Table:

model	no_of_active_vehicles
Volvo VNL	45
Freightliner M2	122
Isuzu N-Series	89

Summary for Developers:

- **Batch Jobs (S3):** Use Spark/Glue to handle the SCD Type 2 logic and deduplication.
- **Scheduling:** Ensure the Fuel Audit runs *after* the Asset History update so it uses the most recent daily_rate and status.
- **Fault Tolerance:** If a job fails and is restarted, ensure it doesn't create duplicate "ARCHIVED" records for the same day.

To implement the **OmniRoute** safety and streaming logic, you must build a stateful system that processes real-time events while interacting with historical driver data.

Below are the key scenarios and technical logic explained through examples.

1. Violation Detection: The Speed & Zone Filter

The streaming engine (e.g., Spark Streaming or Flink) must evaluate every incoming Kafka JSON message against a set of thresholds and a spatial reference file.

Scenario: The High-Speed Intrusion

- **The Reference:** restricted_zones.json contains a polygon for "Zone_Alpha" (e.g., a high-pedestrian area).
- **The Event:** A Kafka message arrives for **VIN-700**:
 - speed: 115 km/h
 - lat/long: Coordinates that fall inside "Zone_Alpha".
- **The Logic:** * The speed is > 110 km/h threshold.

- The coordinates intersect with the restricted zone.
- **Result:** This single message triggers **two** safety flags. However, system logic usually treats a single event window as **one Safety Strike** to prevent over-penalizing for a single second of data.

2. The Penalty System: Dynamic Salary Calculation

Unlike standard payroll, this system requires a "Resultant Table" that calculates a driver's net pay in near real-time based on their behavior.

Scenario: Calculating the "Strike Column"

- **Driver Base Rate:** \$500/day (from the vehicle_assignment.csv).
- **The Event:** Driver-A gets their first Safety Strike of the month.
- **The Logic:** * Deduction = 5% of \$500 = \$25. (Note: The new logistics project uses a 5% penalty compared to the previous 10%).
- **Resultant Table Entry:**

driver_id	base_rate	strike_count	current_adjusted_rate
DRV-A	500	1	475

Developer Note: The current_adjusted_rate must be a separate column so the original base_rate is never lost.

3. Cooldown & Suspension: The "Point of No Return"

This scenario handles the transition between months and identifies high-risk drivers who are no longer eligible for "forgiveness."

Scenario A: Standard Cooldown (The Forgiven)

- **Driver-B** has **3 strikes** on May 31st.
- **The Logic:** On June 1st at 05:00 UTC, the system runs a batch job.
- **Result:** Strikes reset to **0**, and current_adjusted_rate is restored to the base_rate.

Scenario B: Suspension (The Hard Limit)

- **Driver-C** hits their **10th strike** on May 20th.
- **The Logic:** 1. Immediately toggle status to **"SUSPENDED"** (Inactive). 2. When the June 1st Cooldown job runs, it filters for WHERE status != 'SUSPENDED'.
- **Result:** Driver-C stays at 10 strikes, their rate remains penalized (or zeroed), and they are blocked from the fleet.

4. Technical Implementation Guidance for Developers

- **State Management:** Use a stateful streaming approach to keep an "active count" of strikes in memory or a fast-access DB (like Redis or DynamoDB) before flushing to the DWH.
- **Idempotency:** If the monthly cooldown job fails and restarts, it must not reset the same driver twice or accidentally reset a "SUSPENDED" driver.
- **Join Strategy:** In the stream, you must join the Kafka vin with the Asset History table to find the correct driver_id currently assigned to that vehicle.